INSTITUT SUPERIEUR D'INFORMATIQUE

ET DES TECHNIQUES DE COMMUNICATION – HAMMAM SOUSSE

المعهد العالي للإعلامية وتكنولوجيات الاتصال بسوسة

End-of-Studies Internship Report

Presented for obtaining the Bachelor degree in Computer Science
Spécialité : Informatique and Multimédia

CyberScope

Web Application Security Auditing System
Detecting and Analyzing Vulnerabilities
Through Intelligent Automation

ELABORATED BY
MOHAMED GUIZANI

Academic Supervisor: M.Mohamed Amine Ben Amar

Professional Supervisor: M.Yassine Lasoued

Host Company

EY

School Year : 2025/2026

Higher Institute of Computer Science and Communication Technologies ISITCOM Tel/Fax: +216 73 37 15 71 / +216 73 36 44 11

Résumé

Ce projet de fin d'études porte sur la conception et la réalisation de **CyberScope**, une plateforme d'audit de sécurité pour applications web. L'objectif principal est d'automatiser le processus de détection des vulnérabilités tout en améliorant la lisibilité et la pertinence des résultats fournis aux analystes.

La solution proposée combine une interface web développée avec React et TypeScript, un backend Node.js/Express, une base de données PostgreSQL administrée avec Prisma, et un moteur de scan Python basé sur HexStrike. Le système orchestre plusieurs outils de sécurité, notamment Nuclei, SQLMap, Nikto, Nmap, FFUF, Gobuster, Dalfox et Arjun. Les résultats bruts sont ensuite normalisés, dédupliqués, enrichis par des contrôles passifs et associés à des vulnérabilités structurées.

CyberScope intègre aussi des fonctionnalités d'authentification, d'historique des scans, de profils de cible, de diagnostics du pipeline et de génération de scénarios d'attaque assistée par l'intelligence artificielle via Ollama. L'évaluation a été menée sur des applications vulnérables et des sorties de scan représentatives afin d'analyser la couverture de détection, la réduction du bruit et les limites de l'approche.

Mots-clés : cybersécurité, audit web, vulnérabilités, OWASP, HexStrike, automatisation, React, Node.js, intelligence artificielle.

Abstract

This end-of-studies project presents the design and implementation of **CyberScope**, a web application security auditing platform. The main objective is to automate vulnerability assessment while improving the clarity, relevance, and operational value of the findings delivered to security analysts.

The proposed solution combines a React/TypeScript frontend, a Node.js/Express backend, a PostgreSQL database managed through Prisma, and a Python scanning engine based on HexStrike. CyberScope coordinates multiple security tools, including Nuclei, SQLMap, Nikto, Nmap, FFUF, Gobuster, Dalfox, and Arjun. Raw scanner outputs are normalized, deduplicated, enriched with passive security checks, and mapped to structured vulnerability records.

The platform also provides authentication, user-scoped scan history, scan profiles, target profiling, pipeline diagnostics, and AI-assisted attack scenario generation through Ollama. The evaluation was conducted using intentionally vulnerable applications and representative scan outputs in order to assess detection coverage, noise reduction, result quality, and the limitations of the proposed approach.

Keywords: cybersecurity, web auditing, vulnerability scanning, OWASP, HexStrike, automation, React, Node.js, artificial intelligence.

Dedications

First and foremost, I thank **Allah** for choosing me among thousands to face this trial, which has made me more patient, wiser, and stronger. Despite the back injury that has challenged me for more than a year and a half, during which I could barely sit to work, all praise be to Allah for His guidance and strength.

To my beloved family, who replaced my physical pain with their unwavering support, love, and care:

To **Anissa, my mother**, whose devotion knew no bounds—checking on my condition and illness every single day throughout this difficult period. Your constant care has been my healing.

To **Hassen, my father**, who dedicated himself entirely to ensuring my comfort, doing everything possible to ease my burden. Your sacrifices have not gone unnoticed.

To **Iskander**, who became my backbone and eternal supporter when my own failed me. You carried me through the darkest moments.

To **Sabri**, from whom I drew power and strength when mine was depleted. Your presence gave me energy to continue.

To **Naktel**, the youngest among us, who brought joy and happiness into days filled with pain. Your light brightened my path.

Every line of this report was written through immense pain and is dedicated to all of you. Your love transformed my struggle into strength, and your faith in me made this achievement possible.

Alhamdulillahi rabbil alameen.

Acknowledgments

I would like to express my deepest gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I extend my heartfelt appreciation to **M. Yassine Lassoed**, my professional supervisor, who believed in me and in this project from the very beginning. Despite the challenges faced during this internship, he provided me with essential guidance and constant support, enabling me to bring this project to life. His encouragement and trust in my abilities have been invaluable throughout this journey.

I would also like to sincerely thank **M. Mohamed Amine Ben Amar**, my academic supervisor, for his continuous support and valuable advice. Throughout this challenging period, he offered insightful feedback, motivation, and understanding, which greatly contributed to the successful completion of this work.

I am equally grateful to all the individuals who supported me, directly or indirectly, during the realization of this project. Their encouragement and assistance played a significant role in achieving this accomplishment.

Thank you for making this achievement possible.

Table of Contents

Résumé i

Abstract ii

Dedications iii

Acknowledgements iv

Table of Contents v

List of Figures ix

List of Tables x

List of Abbreviations xi

General Introduction 1

1 General Presentation of the Project 2

1.1 Introduction 2

1.2 Presentation of the Host Organization 2

1.3 Project Context and Problem Statement 3

1.4 Overview of Web Application Security 3

1.4.1 OWASP Top 10 and Modern Threat Landscape 4

1.4.2 Limitations of Traditional Vulnerability Scanners 4

1.5 Study of Existing Solutions 4

1.5.1 Nikto 4

1.5.2 OWASP ZAP 5

1.5.3 Burp Suite 5

1.5.4 Comparative Analysis 5

1.6 Critique of Existing Solutions 6

1.7 Proposed Solution: CyberScope 7

1.8 Adopted Development Methodology 7

1.9 Conclusion 8

2 System Analysis and Requirements Specification 9

2.1 Introduction 9
2.2 Stakeholder Identification 9
2.3 System Overview 9
2.4 Functional Requirements 10
2.4.1 Scan Initiation and Target Submission 10
2.4.2 Intelligent Tool Orchestration 11
2.4.3 Dynamic Application Crawling (BrowserAgent) 11
2.4.4 Vulnerability Detection and Analysis 11
2.4.5 Results Aggregation and Visualization 11
2.5 Non-Functional Requirements 12
2.5.1 Performance and Scalability 12
2.5.2 Security Requirements 12
2.5.3 Usability 12
2.6 Use Case Diagram 12
2.7 Product Backlog 13
2.8 Sprint Planning 14
2.9 Conclusion 15

3 System Design and Architecture 16

3.1 Introduction 16
3.2 Global System Architecture 16
3.3 Detailed Architecture 16
3.3.1 Frontend Architecture (React) 17
3.3.2 Backend Architecture (Node.js) 17
3.3.3 Scanning Engine (Python / HexStrike) 17
3.4 Key Components 18
3.4.1 BrowserAgent 18
3.4.2 Orchestration Engine 18
3.4.3 Vulnerability Analyzer 18
3.5 Data Flow and Processing Pipeline 19
3.6 Scrum Alignment of the Architecture 20
3.7 Conclusion 21

4 System Implementation (Sprint-Based Development) 22

4.1 Introduction 22
4.2 Development Environment 22
4.2.1 Technologies and Frameworks 22
4.2.2 Tools and Security Modules 23

4.3 Sprint 1 — Backend Infrastructure Implementation 23
4.3.1 Sprint Goal 23
4.3.2 Sprint Backlog 24
4.3.3 Implementation 24
4.3.4 Sprint Outcome 24
4.4 Sprint 2 — Scanning Engine and Security Pipeline Implementation 25
4.4.1 Sprint Goal 25
4.4.2 Sprint Backlog 25
4.4.3 Implementation 25
4.4.4 Sprint Outcome 26
4.5 Sprint 3 — Frontend Dashboard Implementation 26
4.5.1 Sprint Goal 26
4.5.2 Sprint Backlog 26
4.5.3 Implementation 26
4.5.4 Sprint Outcome 27
4.6 Sprint 4 — Testing and Experimental Validation Setup 27
4.6.1 Sprint Goal 27
4.6.2 Sprint Backlog 27
4.6.3 Implementation 27
4.6.4 Sprint Outcome 27
4.7 Conclusion 28

5 Results, Evaluation and Discussion 29

5.1 Introduction 29
5.2 Experimental Validation 29
5.2.1 Test Environment and Methodology 29
5.2.2 Case Study: OWASP Juice Shop 29
5.2.3 Case Study: Altoro Mutual 30
5.3 Results Analysis 30
5.3.1 True Positives vs False Positives 30
5.3.2 Detection Coverage 30
5.3.3 System Limitations 30
5.4 Comparative Evaluation with Existing Tools 31
5.5 Proposed Enhancements 31
5.6 Future Work 31
5.7 Conclusion 32

6 Limitations, Perspectives and Future Work 33

6.1 Introduction 33
6.2 Synthesis of Contributions 33
6.3 Comparative Positioning 34
6.4 Identified Limitations 34
6.4.1 Dependency on External Tools 34
6.4.2 Scan Duration and Resource Consumption 34
6.4.3 False Positives and Manual Confirmation 35
6.4.4 Limited Authenticated Target Crawling 35
6.4.5 Reporting and Integration 35
6.5 Proposed Enhancements 35
6.6 Future Research Directions 35
6.7 Conclusion 36

General Conclusion **37**

References **38**

A Technical Appendix **41**
A.1 Main API Endpoints 41
A.2 Environment Variables 41
A.3 Validation Scripts and Artifacts 42

List of Figures

1.1 EY Tunisia 2
1.2 Nikto 5
1.3 OWASP ZAP 5
1.4 Burp Suite 5

2.1 Global Use Case Diagram of CyberScope 13

3.1 Global Architecture of CyberScope 16
3.2 CyberScope Scan Processing Pipeline 20

List of Tables

1.1 Comparative analysis of existing solutions 6
1.2 Synthesis of limitations of existing solutions 7

2.1 Product Backlog of CyberScope 14
2.2 Sprint Planning of CyberScope 15

3.1 Main Architectural Layers of CyberScope 17
3.2 Key Components and Responsibilities 19
3.3 Architecture Alignment with Scrum Sprints 21

4.1 Main Technologies and Frameworks 22
4.2 Integrated Security Tools 23
4.3 Sprint 1 Backlog 24
4.4 Sprint 2 Backlog 25
4.5 Sprint 3 Backlog 26
4.6 Sprint 4 Backlog 27

5.1 Comparative Evaluation of CyberScope 31

6.1 Positioning of CyberScope 34

A.1 Main backend API endpoints 41
A.2 Important backend environment variables 41

List of Abbreviations

API	Application Programming Interface
CSP	Content Security Policy
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
DAST	Dynamic Application Security Testing
DNS	Domain Name System
HSTS	HTTP Strict Transport Security
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SPA	Single Page Application
SQLi	SQL Injection
SSRF	Server-Side Request Forgery
TLS	Transport Layer Security
UI	User Interface
XSS	Cross-Site Scripting

General Introduction

In recent years, the rapid evolution of web technologies has significantly transformed the development and deployment of modern applications. Web applications have become increasingly dynamic, interactive, and complex, relying heavily on client-side rendering and continuous communication with backend services. While these advancements have enhanced user experience, they have also expanded the attack surface and introduced new security challenges.

Web application security has become a critical concern for organizations. Vulnerabilities such as SQL injection, cross-site scripting (XSS), and authentication flaws remain among the most exploited attack vectors, as highlighted by the OWASP Top 10 [1]. These vulnerabilities can lead to severe consequences, including data breaches, financial losses, and reputational damage.

To mitigate these risks, various security scanning tools have been developed, including solutions such as Nikto, OWASP ZAP, and Burp Suite. Although these tools provide valuable capabilities, they exhibit significant limitations. In particular, traditional scanners struggle to effectively analyze modern Single Page Applications (SPAs), where content is dynamically generated through JavaScript. Furthermore, they often produce redundant or low-priority findings, making vulnerability analysis more complex and less efficient.

In this context, there is a growing need for more intelligent and adaptive security auditing systems capable of handling modern web architectures while improving detection accuracy and result relevance.

To address these challenges, this project introduces **CyberScope**, an automated web application security auditing system designed to detect and analyze vulnerabilities through intelligent orchestration and advanced automation techniques. The proposed system integrates multiple security tools within a unified pipeline and enhances their capabilities through a browser-based crawling agent capable of exploring dynamic applications. In addition, an AI-driven correlation engine is employed to normalize, deduplicate, and prioritize detected vulnerabilities, providing more meaningful and actionable insights.

The objective of this work is to design and implement a comprehensive and efficient security auditing platform that improves vulnerability detection coverage and adapts to the complexity of modern web environments.

Chapter 1

General Presentation of the Project

1.1 Introduction

This chapter focuses on the general context of the project. It presents the background of the work, the security challenges faced by modern web applications, the limitations of existing solutions, and the motivation behind the proposed system, **CyberScope**.

1.2 Presentation of the Host Organization

Ernst & Young (EY) [2] is one of the world's leading professional services firms, recognized globally for its expertise in assurance, consulting, strategy, tax, and transaction services. Founded in 1989 through the merger of Ernst & Whinney and Arthur Young, EY operates in more than 150 countries and employs hundreds of thousands of professionals worldwide.

In Tunisia, EY plays a key role in supporting organizations in their digital transformation and technological innovation. The firm assists both public and private institutions in addressing complex business challenges by providing tailored consulting services and advanced technical solutions.

EY Tunisia offers a wide range of services, including IT consulting, cybersecurity, risk management, and digital innovation. In particular, the firm contributes to strengthening information systems security by identifying vulnerabilities and implementing robust protection mechanisms.

Through its global network and local expertise, EY promotes best practices in cybersecurity and technological development. Its commitment to quality and innovation makes it an ideal environment for the development of advanced solutions such as automated web security auditing systems.

Figure 1.1: EY Tunisia

1.3 Project Context and Problem Statement

With the increasing reliance on web applications in critical domains such as finance, e-commerce, and healthcare, ensuring their security has become a major challenge. Modern applications are more dynamic and complex, which significantly increases their exposure to various cyber threats such as SQL injection, cross-site scripting, and authentication vulnerabilities.

Despite the availability of several security tools, existing solutions often fail to effectively analyze modern web applications, especially those based on dynamic architectures. These tools may miss critical vulnerabilities, generate redundant results, or require significant manual intervention, which reduces their efficiency and reliability.

In this context, the main problem addressed in this project is the lack of an intelligent and automated solution capable of efficiently auditing modern web applications while improving vulnerability detection accuracy and reducing false positives.

The objective of this project is therefore to design and develop **CyberScope**, an advanced web application security auditing system that enhances vulnerability detection through intelligent orchestration, automation, and improved analysis of security findings.

1.4 Overview of Web Application Security

Web application security refers to the set of measures and practices implemented to protect web applications from vulnerabilities and cyber threats. As web applications become increasingly central to business operations and data management, ensuring their security is essential to preserve the confidentiality, integrity, and availability of information.

Modern web applications are exposed to a wide range of attacks that exploit weaknesses in their design, implementation, or configuration. These attacks may target input validation, authentication mechanisms, session management, or communication between client and server. As a result, a single vulnerability can compromise the entire system and lead to severe consequences such as data breaches, financial loss, or reputational damage.

To mitigate these risks, security must be integrated throughout the entire development lifecycle. This includes secure coding practices, regular vulnerability assessments, and the use of automated security tools capable of identifying potential weaknesses before they are exploited.

In this context, standards such as the OWASP Top 10 provide valuable guidance by identifying the most critical web application security risks [1]. These references help developers and security professionals prioritize their efforts and implement effective protection strategies.

1.4.1 OWASP Top 10 and Modern Threat Landscape

The OWASP Top 10 is a widely recognized standard that identifies the most critical security risks affecting web applications [1]. It provides a prioritized list of vulnerabilities such as injection attacks, broken authentication, sensitive data exposure, and cross-site scripting.

These vulnerabilities represent the most common attack vectors exploited by attackers in real-world scenarios. As web technologies evolve, the threat landscape continues to expand, with modern applications introducing new risks related to client-side execution, APIs, and complex system interactions.

By highlighting the most significant threats, the OWASP Top 10 helps developers and security professionals focus on the most impactful vulnerabilities and adopt effective mitigation strategies.

1.4.2 Limitations of Traditional Vulnerability Scanners

Traditional vulnerability scanners are widely used to detect security flaws in web applications through automated analysis and predefined signatures. While these tools are effective for identifying known vulnerabilities, they present several limitations when applied to modern web environments.

In particular, they often struggle to analyze dynamic applications, especially those based on client-side rendering and asynchronous communication. As a result, important attack surfaces may remain unexplored. Additionally, these tools may generate a high number of false positives and redundant findings, which complicates the analysis process and reduces their overall efficiency.

These limitations highlight the need for more advanced and intelligent approaches to web application security auditing.

1.5 Study of Existing Solutions

Several tools have been developed to identify vulnerabilities in web applications and assist security professionals in detecting potential threats. These tools provide different approaches to vulnerability scanning, ranging from simple automated checks to advanced interactive analysis. Among the most widely used solutions are Nikto, OWASP ZAP, and Burp Suite.

1.5.1 Nikto

Nikto is an open-source web server scanner designed to detect known vulnerabilities, outdated software, and misconfigurations in web servers [3]. It performs comprehensive checks against a large database of known issues and is commonly used for quick security assessments.

However, Nikto is limited to server-side analysis and does not effectively handle

modern dynamic web applications, which reduces its applicability in complex environments.

Figure 1.2: Nikto

1.5.2 OWASP ZAP

OWASP ZAP is a widely used open-source security testing tool that provides automated scanning as well as manual testing capabilities [4]. It is capable of detecting a variety of vulnerabilities such as injection flaws and security misconfigurations.

Despite its flexibility, ZAP may require proper configuration and user expertise to achieve optimal results, and it can produce a significant number of false positives in complex applications.

Figure 1.3: OWASP ZAP

1.5.3 Burp Suite

Burp Suite is a professional web security testing platform offering a comprehensive set of tools for vulnerability detection and exploitation [5]. It allows detailed analysis of HTTP requests and responses and supports advanced manual testing techniques.

While powerful, Burp Suite is more complex to use and often requires significant manual intervention, which can make the testing process time-consuming.

Figure 1.4: Burp Suite

1.5.4 Comparative Analysis

To better evaluate the strengths and limitations of the studied tools, a comparative analysis is presented below. Although Nikto, OWASP ZAP, and Burp Suite are widely

used in web application security testing, each of them addresses only part of the overall auditing process.

Nikto is well suited for rapid server-side checks, OWASP ZAP provides a balanced approach between automation and usability, and Burp Suite offers advanced manual testing capabilities. However, none of these tools fully combines intelligent automation, deep dynamic exploration, and effective reduction of noisy findings.

Table 1.1: Comparative analysis of existing solutions

Criteria	**Nikto**	**OWASP ZAP**	**Burp Suite**
Type	Open-source server scanner	Open-source proxy and scanner	Professional security testing suite
Main approach	Signature-based scanning	Automated and manual testing	Advanced manual and automated testing
Dynamic application support	Limited	Partial	Good
Ease of use	Easy	Moderate	Complex
False positives	High	Medium	Low to medium
Manual testing support	No	Yes	Advanced
Automation level	High	High	Medium
Best suited for	Quick server checks	General web security testing	Deep vulnerability analysis

This comparison shows that existing tools provide useful but incomplete capabilities. Their limitations in terms of automation, adaptability, and dynamic analysis justify the need for a more integrated and intelligent solution such as **CyberScope**.

1.6 Critique of Existing Solutions

Although existing tools such as Nikto, OWASP ZAP, and Burp Suite provide valuable support for vulnerability detection, they still present several limitations when applied to modern web applications.

These limitations are mainly related to the lack of support for dynamic applications, the generation of false positives, and the need for manual intervention. In addition, the absence of intelligent correlation between detected vulnerabilities reduces the clarity and efficiency of the analysis process.

Table 1.2: Synthesis of limitations of existing solutions

Limitation	Description	Impact
Limited support for dynamic applications	Difficulty handling JavaScript-based applications	Incomplete vulnerability detection
High false positives	Redundant or irrelevant findings	Time-consuming analysis
Lack of intelligent correlation	No prioritization of vulnerabilities	Reduced clarity of results
Manual intervention required	Dependence on user expertise	Low scalability
Fragmented tool usage	Multiple tools used separately	Complex workflow

These limitations highlight the need for a more advanced solution capable of combining automation, accuracy, and adaptability.

1.7 Proposed Solution: CyberScope

To overcome the limitations identified in existing solutions, this project proposes **CyberScope**, an intelligent web application security auditing system.

CyberScope is based on a unified architecture that integrates multiple security tools into a single automated pipeline. It introduces a browser-based crawling mechanism capable of exploring modern dynamic applications and interacting with client-side components.

In addition, the system incorporates an intelligent analysis layer responsible for normalizing, deduplicating, and prioritizing vulnerabilities. This significantly reduces noise and improves the relevance of the results.

By combining automation, dynamic exploration, and intelligent processing, CyberScope aims to provide a more efficient and scalable approach to web application security auditing.

1.8 Adopted Development Methodology

To ensure an efficient and flexible development process, the Scrum methodology was adopted for the realization of this project. Scrum is an agile framework that promotes iterative development, incremental delivery, continuous feedback, and adaptability.

In the context of this project, Scrum was adapted to a final-year internship environment. The work was organized into four main sprints, each focusing on a specific objective: backend infrastructure, scanning engine integration, frontend dashboard development, and experimental validation.

The sprint planning phase was used to define the main objectives and select the relevant user stories from the product backlog. At the end of each sprint, the implemented features were reviewed and adjusted according to technical constraints and project priorities.

This sprint-based organization allowed the project to evolve progressively while maintaining a clear link between requirements, architecture, implementation, and validation. The adoption of Scrum contributed to improving productivity, organization, and adaptability, which were essential for the successful development of **CyberScope**.

1.9 Conclusion

This chapter introduced the general context of the project by highlighting the importance of web application security and the challenges posed by modern dynamic applications. It also presented a study of existing solutions and identified their main limitations.

Based on this analysis, the need for a more advanced and intelligent approach was established, leading to the proposal of **CyberScope**. The next chapter will focus on the analysis of the system requirements and the initialization of the Scrum process.

Chapter 2

System Analysis and Requirements Specification

2.1 Introduction

This chapter presents the analysis of the proposed system, **CyberScope**. It identifies the main stakeholders involved in the platform and defines the functional and non-functional requirements that guide its development.

It also introduces the Scrum initialization phase by defining the product backlog and the sprint planning adopted during the development process.

2.2 Stakeholder Identification

The development and use of **CyberScope** involve several stakeholders, each interacting with the system according to specific roles.

The primary stakeholder is the **Security Analyst**, who represents the main user of the platform. This actor is responsible for submitting target URLs, initiating automated scans, and analyzing the results through the system dashboard. The analyst reviews detected vulnerabilities, examines their severity levels, and analyzes the evidence generated during the scanning process.

The **System Administrator** is responsible for deploying and maintaining the platform. This includes managing system configurations, ensuring the availability of integrated scanning tools, including the Python-based scanning engine and its integrated tools, and supervising backend services to guarantee proper system operation.

Academic and professional supervisors are considered indirect stakeholders. They are involved in evaluating the project, validating its objectives, and ensuring that the developed solution meets both technical and academic requirements.

Finally, the **target application owner** represents an external stakeholder. Although they do not directly interact with the system, their web applications are analyzed by CyberScope. They benefit from the identification of vulnerabilities and the actionable results produced by the platform.

2.3 System Overview

CyberScope is an intelligent web application security auditing platform designed to automate the detection and analysis of vulnerabilities in modern web environments. It aims to provide a unified solution that combines multiple security tools with advanced analysis capabilities.

The system is structured around three main components: a web-based user interface, an orchestration backend, and a scanning engine. The user interacts with the platform through a dashboard that allows the submission of target URLs, the initiation of scans, and the visualization of results.

When a scan is launched, the backend coordinates the execution of multiple security tools through an automated pipeline. The scanning engine performs the analysis by combining traditional vulnerability scanning techniques with dynamic exploration of web applications. The system also generates a **target profile** characterizing the attack surface, technology stack, and exposed services of the analyzed application.

The collected results are then processed, aggregated, and presented to the user in a structured and readable format. This global architecture enables CyberScope to handle complex and dynamic applications while improving the accuracy and relevance of detected vulnerabilities.

2.4 Functional Requirements

The functional requirements define the core services that CyberScope must provide to ensure efficient and automated web application security auditing.

The system must allow users to initiate security scans on target web applications and manage the entire scanning lifecycle. It should support the integration of multiple security tools and coordinate their execution through an automated and intelligent pipeline.

CyberScope must also be capable of analyzing modern web applications by incorporating dynamic exploration mechanisms, enabling deeper coverage of client-side behavior and hidden attack surfaces. In addition, the system must detect and analyze vulnerabilities by combining the outputs of different tools and presenting them in a structured manner.

Finally, the platform must provide a clear and intuitive visualization of scan results, allowing users to easily interpret findings, assess their severity, and make informed decisions regarding security improvements.

2.4.1 Scan Initiation and Target Submission

The system must allow the user to submit a target web application in the form of a URL and initiate a security scan. This functionality represents the starting point of the auditing process.

Once a target is provided, the system must validate the input and automatically trigger the scanning workflow. The platform should manage the execution of the scan without requiring manual intervention, ensuring a seamless and efficient user experience.

In addition, the system must support the management of multiple scan requests,

allowing users to monitor ongoing scans and access previously executed ones.

2.4.2 Intelligent Tool Orchestration

The system must ensure the coordinated execution of multiple security tools within a unified scanning pipeline. This functionality enables CyberScope to perform comprehensive vulnerability assessments by combining the strengths of different tools.

The platform should automatically select and execute the appropriate tools based on the characteristics of the target application. It must manage the execution flow, ensuring that tools operate efficiently and without unnecessary redundancy.

By orchestrating multiple tools in an intelligent manner, the system improves detection coverage and enhances the overall quality of the security analysis.

2.4.3 Dynamic Application Crawling (BrowserAgent)

The system must support the exploration of modern web applications through a dynamic crawling mechanism. This functionality is essential for analyzing Single Page Applications and client-side interactions that are not accessible through traditional scanning techniques.

To achieve this, the platform integrates a browser-based agent capable of interacting with web pages, triggering user actions, and capturing underlying network requests. This allows the system to uncover hidden endpoints and improve the detection of vulnerabilities in dynamic environments.

2.4.4 Vulnerability Detection and Analysis

The system must be capable of detecting security vulnerabilities by analyzing the outputs generated by multiple integrated tools. It should identify common web application vulnerabilities such as SQL injection, cross-site scripting (XSS), path traversal, and security misconfigurations.

In addition, the platform must analyze the detected findings to provide meaningful insights, allowing users to understand the nature and potential impact of each vulnerability.

2.4.5 Results Aggregation and Visualization

The system must process and aggregate scan results obtained from different tools to eliminate redundancy and improve clarity. This includes organizing findings and applying deduplication mechanisms such as hash-based comparison to reduce duplicated results.

The results must be presented through a structured and intuitive interface, enabling users to easily interpret the findings and make informed decisions regarding security improvements.

2.5 Non-Functional Requirements

In addition to functional requirements, CyberScope must satisfy several non-functional requirements to ensure reliability, efficiency, and usability.

2.5.1 Performance and Scalability

The system should provide acceptable response times during scan execution and result processing. Scan initiation should remain within acceptable response times, typically within a few seconds, ensuring a responsive user experience.

It must also be scalable to handle multiple scan requests and support the analysis of large and complex web applications.

2.5.2 Security Requirements

The platform must ensure secure access control and protect sensitive data related to scan results and system operations. It should implement secure authentication mechanisms, such as token-based authentication, and safeguard communications between system components.

2.5.3 Usability

The user interface must be intuitive and easy to use for security analysts. The visualization of results should be clear and well-structured, facilitating the interpretation of findings and improving user experience.

2.6 Use Case Diagram

The use case diagram illustrates the main interactions between the primary actor, the security analyst, and the CyberScope system. It highlights the core functionalities of the platform, including submitting targets, launching scans, and analyzing results.

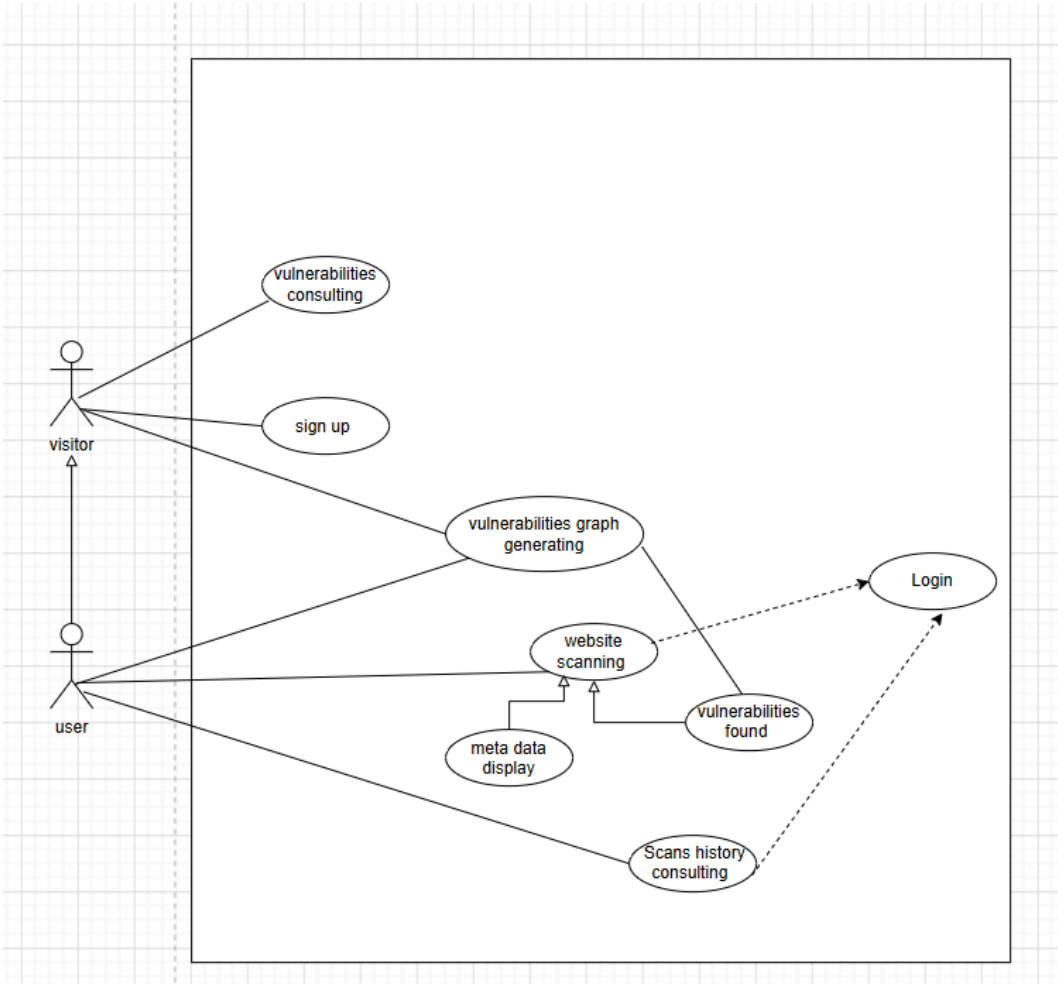

Figure 2.1: Global Use Case Diagram of CyberScope

2.7 Product Backlog

The product backlog defines the set of functionalities that **CyberScope** must implement. It is expressed in the form of user stories that describe the system from the perspective of the end user.

Each user story is assigned a priority level and an estimated complexity. The prioritization helps determine the order of implementation, while the complexity estimation supports sprint planning. The complexity estimation was carried out using a simplified T-shirt sizing approach (S, M, L, XL), inspired by agile estimation techniques such as Planning Poker.

Table 2.1: Product Backlog of CyberScope

ID	User Story	Priority	Complexity
US1	As a security analyst, I want to submit a target URL in order to launch a security scan.	High	S
US2	As a security analyst, I want to monitor the progress of a scan in real time.	High	M
US3	As a security analyst, I want the system to automatically detect vulnerabilities in web applications.	High	L
US4	As a security analyst, I want the system to analyze dynamic web applications using a browser-based crawler.	High	XL
US5	As a security analyst, I want to view structured scan results with severity levels and evidence.	High	M
US6	As a system administrator, I want to manage system configuration and integrated tools.	Medium	M
US7	As a user, I want to access previous scans and their results.	Medium	S
US8	As a system, I want to generate a target profile describing the attack surface and technologies used.	High	L
US9	As a system, I want to reduce duplicate findings using intelligent aggregation mechanisms.	High	M
US10	As a security analyst, I want to receive AI-powered remediation advice and business impact reports for each finding.	High	M

2.8 Sprint Planning

After defining the product backlog, the user stories were organized into four main sprints. Each sprint focuses on a specific development objective and contributes incrementally to the construction of the final system.

Table 2.2: Sprint Planning of CyberScope

Sprint	**Goal**	**Related User Stories**
Sprint 1	Build the backend infrastructure, database schema, authentication, and connection with the Python scanning engine.	US1, US6, US7
Sprint 2	Implement the scanning engine, tool orchestration, target profiling, dynamic crawling, and vulnerability detection pipeline.	US3, US4, US8, US9
Sprint 3	Develop the frontend dashboard, scan interface, result visualization, scan history, and target profile display.	US2, US5, US7
Sprint 4	Validate the system through experimental testing, evaluate results, and assess the quality of detected findings.	US3, US5, US9, US10

This sprint planning provides a clear roadmap for the implementation phase. It ensures traceability between the requirements, the Scrum process, and the technical development of the platform.

2.9 Conclusion

This chapter presented the analysis of the CyberScope system by identifying its stakeholders and defining both functional and non-functional requirements. It also introduced the Scrum initialization phase through the product backlog and sprint planning.

These elements establish a clear understanding of the system's objectives and provide a solid foundation for the design and architecture presented in the next chapter.

Chapter 3

System Design and Architecture

3.1 Introduction

This chapter presents the design and architecture of the proposed solution, **CyberScope**. It describes the global structure of the system, its main components, and the data flow followed during the scanning process.

The objective of this chapter is to define how the different parts of the platform interact before moving to the implementation phase. The design choices presented here also prepare the sprint-based implementation described in the next chapter.

3.2 Global System Architecture

CyberScope follows a multi-layer architecture composed of three main parts: the frontend dashboard, the Node.js orchestration backend, and the Python-based scanning engine. This separation improves modularity, maintainability, and scalability.

The frontend provides the user interface through which the security analyst submits targets, launches scans, and views results. The backend manages authentication, scan lifecycle, database operations, and communication with the scanning engine. The Python engine is responsible for target profiling, tool orchestration, dynamic crawling, and vulnerability discovery.

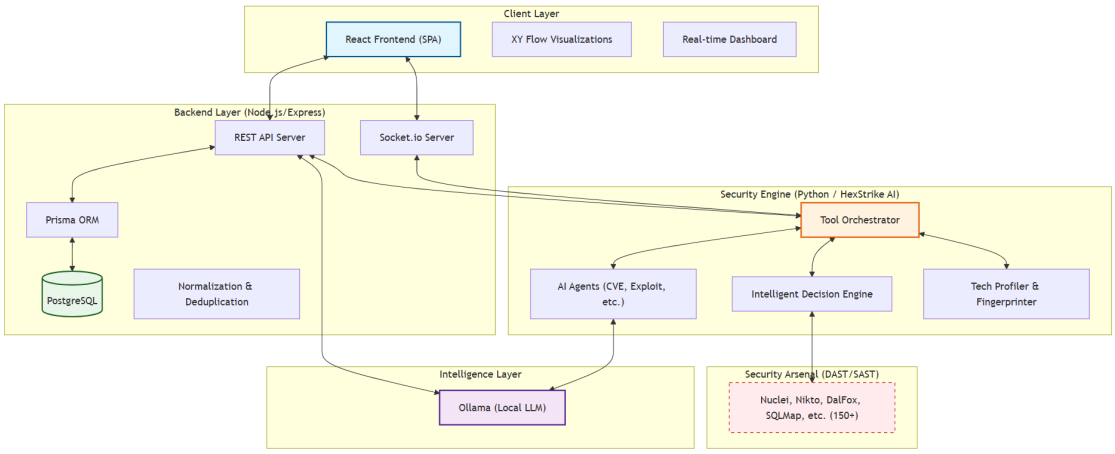

Figure 3.1: Global Architecture of CyberScope

3.3 Detailed Architecture

The detailed architecture of CyberScope is organized around three complementary layers. Each layer has a specific responsibility and communicates with the others through well-defined interfaces.

3.3.1 Frontend Architecture (React)

The frontend of CyberScope is developed using React and TypeScript [6]. It provides the main interaction point between the user and the platform. Through the dashboard, the security analyst can submit a target URL, monitor scan execution, view target profile information, and analyze the detected vulnerabilities.

The frontend includes several main pages such as the authentication pages, the scanner interface, the scan history page, and the scan details page. These interfaces are designed to provide a clear visualization of results and support efficient decision-making.

3.3.2 Backend Architecture (Node.js)

The backend is implemented using Node.js and Express [7]. It acts as the orchestration layer between the frontend, the database, and the Python scanning engine. Its responsibilities include receiving scan requests, managing authentication, storing scan data, and exposing API endpoints to the frontend.

The backend also includes services responsible for scan management, communication with the HexStrike engine, vulnerability matching, normalization, and attack path processing. Data persistence relies on PostgreSQL and Prisma, which respectively provide relational storage and object-relational mapping capabilities [8, 9]. This modular organization makes the system easier to maintain and extend.

3.3.3 Scanning Engine (Python / HexStrike)

The scanning engine is implemented as a Python-based server. It represents the core security analysis layer of CyberScope. Its role is to analyze the submitted target, build a target profile, select appropriate tools, and execute the scanning pipeline.

The engine integrates several security tools such as Nuclei, SQLMap, Nikto, Gobuster, Dalfox, Arjun, Nmap, and other reconnaissance or vulnerability detection modules [10, 11, 3, 12]. It also includes the BrowserAgent component, which improves analysis coverage for modern dynamic web applications.

Table 3.1: Main Architectural Layers of CyberScope

Layer	**Role**	**Main Technologies**
Frontend Dashboard	User interaction, scan submission, result visualization	React, TypeScript, Vite
Orchestration Backend	API management, authentication, scan lifecycle, database access	Node.js, Express, PostgreSQL
Scanning Engine	Target profiling, tool selection, crawling, vulnerability detection	Python, Flask, Selenium, mitmproxy, security tools

3.4 Key Components

The architecture of CyberScope is based on several key components that provide the main intelligence and automation capabilities of the system.

3.4.1 BrowserAgent

The BrowserAgent is one of the most important components of CyberScope. It is designed to analyze modern web applications that rely heavily on JavaScript and client-side rendering.

Unlike traditional crawlers, the BrowserAgent uses a real browser environment to interact with the application. It can trigger user actions, navigate through dynamic pages, and capture network requests generated by the frontend. This allows CyberScope to discover hidden API endpoints and parameters that static scanners may miss.

The BrowserAgent also supports traffic interception through mitmproxy, enabling the system to observe requests produced during navigation and use them as input for deeper vulnerability testing.

3.4.2 Orchestration Engine

The orchestration engine is responsible for coordinating the execution of multiple security tools. Instead of launching all tools randomly, the system analyzes the target and selects the appropriate tools according to its characteristics.

This component improves scanning efficiency by avoiding unnecessary execution and organizing the workflow between reconnaissance, crawling, vulnerability detection, and result collection.

3.4.3 Vulnerability Analyzer

The vulnerability analyzer processes the raw findings produced by different tools. Its role is to normalize the results, reduce duplicates, assign severity levels, and match findings against known vulnerability definitions.

This component improves the readability and usefulness of the final results. Instead of presenting raw outputs directly to the user, CyberScope provides structured findings with severity, evidence, and contextual information.

Table 3.2: Key Components and Responsibilities

Component	**Responsibility**
BrowserAgent	Explores dynamic applications, interacts with client-side elements, and captures API requests.
Orchestration Engine	Selects and coordinates security tools according to the target profile.
Vulnerability Analyzer	Normalizes, deduplicates, prioritizes, and structures detected findings.
Target Profile Module	Characterizes the target attack surface, technologies, services, and risk indicators.
Dashboard	Displays scans, findings, target metadata, and severity distribution to the user.

3.5 Data Flow and Processing Pipeline

The CyberScope scanning process follows a structured pipeline. The user first submits a target URL from the frontend dashboard. The backend receives the request and creates a scan record before forwarding the target to the Python scanning engine.

The scanning engine analyzes the target and generates a target profile. Based on this profile, the orchestration engine selects the appropriate security tools and launches them. The raw results are then returned to the backend, where they are normalized, deduplicated, matched with known vulnerability definitions, and stored in the database.

Finally, the frontend retrieves the processed results and displays them in a clear format, including vulnerability details, severity levels, evidence, and scan diagnostics.

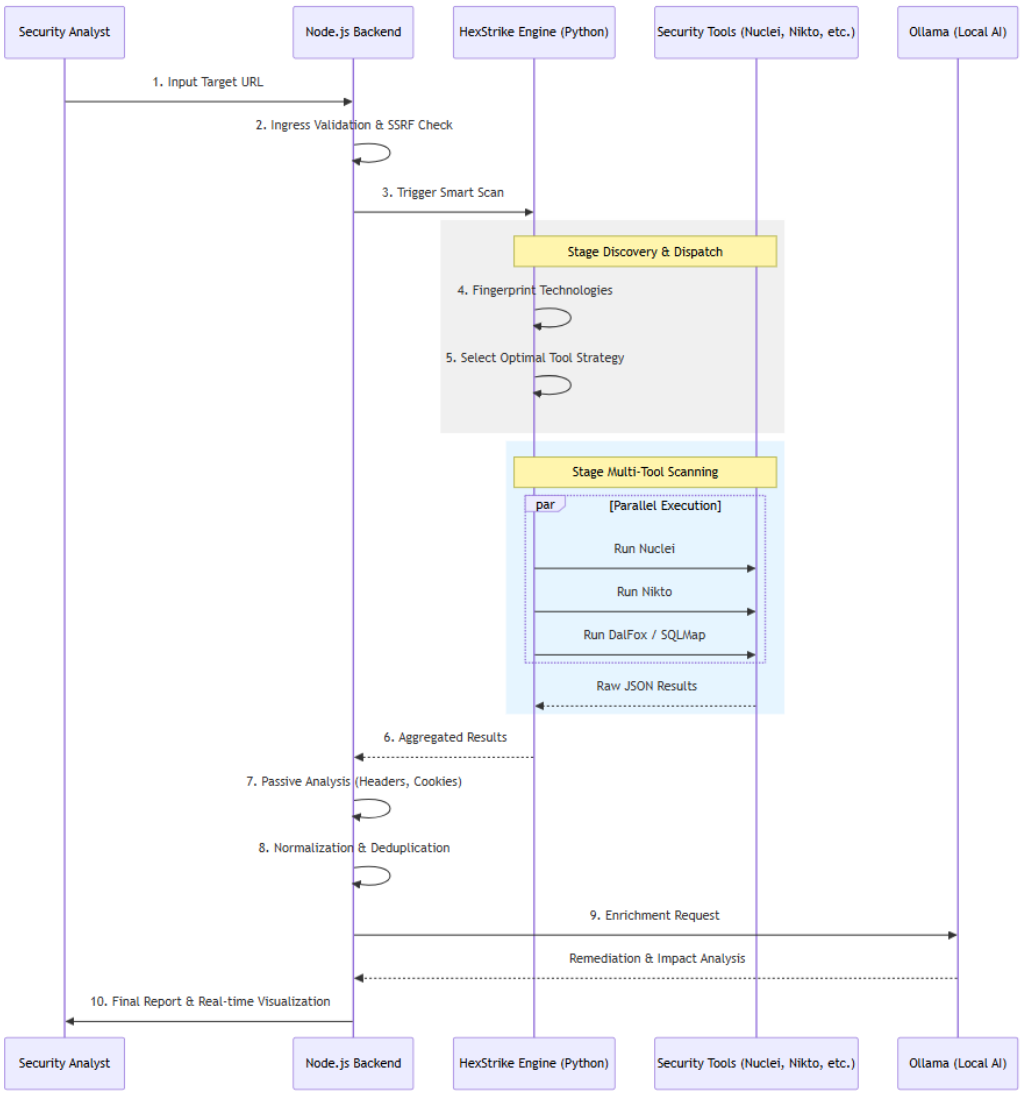

Figure 3.2: CyberScope Scan Processing Pipeline

3.6 Scrum Alignment of the Architecture

The architecture of CyberScope was designed to support the sprint-based implementation process. Each sprint focuses on a specific layer or functional block of the system, allowing progressive development and validation.

Table 3.3: Architecture Alignment with Scrum Sprints

Sprint	**Architectural Focus**	**Main Deliverables**
Sprint 1	Backend infrastructure and database layer	API structure, authentication, PostgreSQL schema, Python engine connection
Sprint 2	Scanning engine and security pipeline	Tool integration, target profiling, intelligent orchestration, BrowserAgent, normalization engine
Sprint 3	Frontend dashboard and user interaction	Scanner page, scan history, results display, target profile visualization
Sprint 4	Validation and evaluation	Case studies, scan evaluation, result analysis, limitations and improvements

This alignment ensures that each sprint contributes to a clear architectural objective. It also helps maintain traceability between the design decisions and the implementation process.

3.7 Conclusion

This chapter presented the design and architecture of CyberScope. It described the global architecture, the main technical layers, the key components, and the scan processing pipeline.

The proposed architecture provides a solid foundation for an intelligent and automated web application security auditing system. The next chapter will focus on the sprint-based implementation of these architectural components.

Chapter 4

System Implementation (Sprint-Based Development)

4.1 Introduction

This chapter presents the implementation of **CyberScope** through a sprint-based development approach. Each sprint corresponds to a coherent implementation phase and contributes incrementally to the construction of the final system.

The chapter focuses on the main technical achievements of each sprint, including the backend infrastructure, the scanning engine, the frontend dashboard, and the experimental validation setup.

4.2 Development Environment

The development environment was selected to support the different layers of CyberScope, including the frontend dashboard, the backend orchestration layer, the Python-based scanning engine, and the database.

4.2.1 Technologies and Frameworks

Table 4.1: Main Technologies and Frameworks

Technology	Usage
React / TypeScript	Development of the frontend dashboard and interactive user interfaces.
Node.js / Express	Backend orchestration, API routing, authentication, and scan lifecycle management.
Python / Flask	Implementation of the scanning engine and security tool orchestration.
PostgreSQL	Storage of users, scans, vulnerabilities, and analysis results.
Selenium / mitmproxy	Dynamic crawling, browser automation, and network traffic interception.

The selected stack reflects the implemented project structure. React and TypeScript were used for the client interface, Express for REST routing, PostgreSQL and Prisma for persistence, and JWT for stateless authentication [6, 7, 8, 9, 13].

4.2.2 Tools and Security Modules

CyberScope integrates several security tools and modules in order to improve vulnerability detection coverage. These tools are coordinated by the scanning engine and used according to the target profile.

Table 4.2: Integrated Security Tools

Tool	**Role**
Nuclei	Template-based vulnerability detection.
SQLMap	Automated SQL injection testing.
Nikto	Web server vulnerability and misconfiguration scanning.
Nmap	Service discovery and port scanning.
Gobuster / FFUF	Directory and endpoint discovery.
Dalfox	Cross-site scripting detection.
Arjun	Parameter discovery.

The role of these tools is not identical. Nuclei is mainly used for template-based detection, SQLMap for SQL injection testing, Nikto for web server checks, and Nmap for network and service discovery [10, 11, 3, 12]. CyberScope uses their outputs as inputs to its own normalization and analysis pipeline.

4.3 Sprint 1 — Backend Infrastructure Implementation

4.3.1 Sprint Goal

The goal of the first sprint was to establish the backend foundation of CyberScope. This included setting up the server structure, designing the database schema, implementing authentication, and preparing the communication layer with the Python scanning engine.

4.3.2 Sprint Backlog

Table 4.3: Sprint 1 Backlog

ID	Task	Related User Story
T1.1	Create backend project structure using Node.js and Express.	US1
T1.2	Design PostgreSQL schema for scans, users, and vulnerabilities.	US7
T1.3	Implement authentication and protected routes.	US6
T1.4	Create APIs for scan creation, scan history, and scan details.	US1, US7
T1.5	Establish communication with the Python scanning engine.	US1

4.3.3 Implementation

During this sprint, the backend was implemented as the central orchestration layer of the system. It exposes REST APIs used by the frontend to initiate scans, retrieve scan history, and access detailed scan results.

The database schema was designed to store scan metadata, target information, vulnerability findings, severity levels, and execution status. This structure enables the system to keep track of previous scans and provide historical visibility to the user.

Authentication mechanisms were also introduced to protect access to the platform and ensure that scan data remains available only to authorized users. The implementation uses password hashing and JWT-based sessions, which allows the backend to identify the user associated with each scan [13].

The backend also validates target URLs before launching scans. This step is important because security scanners can otherwise be abused to reach internal services. The implemented checks follow the same risk principle described in SSRF prevention guidance: private, loopback, and reserved addresses must not be reachable through a user-controlled URL [14].

4.3.4 Sprint Outcome

At the end of Sprint 1, CyberScope had a functional backend foundation capable of receiving scan requests, managing scan records, communicating with the Python engine, and storing results in the database.

4.4 Sprint 2 — Scanning Engine and Security Pipeline Implementation

4.4.1 Sprint Goal

The objective of Sprint 2 was to implement the core scanning engine of CyberScope. This sprint focused on target profiling, intelligent tool orchestration, dynamic crawling, vulnerability detection, normalization, and deduplication.

4.4.2 Sprint Backlog

Table 4.4: Sprint 2 Backlog

ID	Task	Related User Story
T2.1	Implement target profiling logic.	US8
T2.2	Integrate security tools such as Nuclei, SQLMap, Nikto, and Nmap.	US3
T2.3	Implement intelligent tool selection and orchestration.	US3, US8
T2.4	Develop BrowserAgent for dynamic application crawling.	US4
T2.5	Implement normalization and deduplication mechanisms.	US9

4.4.3 Implementation

The Python scanning engine represents the core intelligence layer of CyberScope. It analyzes the submitted target, extracts relevant metadata, generates a target profile, and selects the appropriate security tools to execute.

The tool orchestration process allows CyberScope to combine the strengths of multiple scanners. For example, SQLMap is used for SQL injection testing, Nuclei for template-based detection, Nikto for web server analysis, and Dalfox for cross-site scripting detection.

A major component implemented during this sprint is the BrowserAgent. This component interacts with modern web applications using a real browser environment and captures requests generated by client-side interactions. This improves the coverage of Single Page Applications and helps discover hidden endpoints.

The raw findings generated by the tools are processed by a normalization layer. This layer structures the results, maps severity levels, and applies deduplication mechanisms to reduce duplicated findings.

4.4.4 Sprint Outcome

At the end of Sprint 2, CyberScope was able to profile targets, orchestrate multiple security tools, explore dynamic applications, and produce normalized vulnerability findings.

4.5 Sprint 3 — Frontend Dashboard Implementation

4.5.1 Sprint Goal

The goal of Sprint 3 was to implement the frontend dashboard and provide the user with a clear interface for launching scans, monitoring their progress, and analyzing the results.

4.5.2 Sprint Backlog

Table 4.5: Sprint 3 Backlog

ID	Task	Related User Story
T3.1	Create the scanner page for target submission.	US1
T3.2	Implement scan monitoring interface.	US2
T3.3	Display structured vulnerability results.	US5
T3.4	Create scan history and scan details pages.	US7
T3.5	Display target profile metadata and severity information.	US5, US8

4.5.3 Implementation

The frontend was developed using React and TypeScript. It provides the main interaction layer between the security analyst and CyberScope. TanStack Query is used to manage API requests and asynchronous scan data on the client side [15].

The scanner interface allows the user to submit a target URL and launch a scan. The scan history page enables the user to access previous scans, while the scan details page presents vulnerabilities in a structured format.

The dashboard also displays important metadata such as target profile information, vulnerability severity, scan diagnostics, and evidence associated with findings. The attack path interface uses React Flow to render AI-generated attack scenarios as interactive graphs [16]. This improves the readability of the results and supports decision-making.

4.5.4 Sprint Outcome

At the end of Sprint 3, the user interface was able to support the complete scan lifecycle, from target submission to result visualization.

4.6 Sprint 4 — Testing and Experimental Validation Setup

4.6.1 Sprint Goal

The goal of Sprint 4 was to prepare and execute the validation phase of CyberScope. This sprint focused on testing the platform against vulnerable web applications and collecting data for the evaluation chapter.

4.6.2 Sprint Backlog

Table 4.6: Sprint 4 Backlog

ID	Task	Related User Story
T4.1	Prepare vulnerable test targets.	US3
T4.2	Execute scans against OWASP Juice Shop.	US3, US4
T4.3	Execute scans against Altoro Mutual.	US3
T4.4	Analyze findings and classify results.	US5, US9
T4.5	Prepare data for evaluation and comparison.	US10

4.6.3 Implementation

During this sprint, the system was tested against vulnerable applications in order to evaluate its scanning capabilities. OWASP Juice Shop was selected to test dynamic and modern web application behavior, while Altoro Mutual was used to evaluate traditional web application vulnerabilities.

The objective was not only to detect vulnerabilities but also to observe how CyberScope handles scan execution, target profiling, dynamic exploration, result normalization, and visualization.

4.6.4 Sprint Outcome

At the end of Sprint 4, CyberScope was ready for evaluation. The collected results are analyzed in the next chapter to assess detection coverage, result quality, and system limitations.

4.7 Conclusion

This chapter presented the implementation of CyberScope through four Scrum sprints. It described the development environment, backend infrastructure, scanning engine, frontend dashboard, and testing setup.

The next chapter evaluates the results obtained during experimental validation and discusses the effectiveness, limitations, and possible improvements of the proposed system.

Chapter 5

Results, Evaluation and Discussion

5.1 Introduction

This chapter presents the evaluation of **CyberScope** based on the experimental validation performed during the final sprint. It analyzes the obtained results, discusses the effectiveness of the system, and identifies its limitations and future improvements.

Unlike the previous chapter, which focused on implementation, this chapter focuses on evaluating whether the developed system meets its objectives in terms of vulnerability detection, result clarity, and support for modern web applications.

5.2 Experimental Validation

The experimental validation was conducted using intentionally vulnerable web applications. These targets were selected because they contain known weaknesses and are commonly used to evaluate web application security tools.

5.2.1 Test Environment and Methodology

The validation process consisted of launching scans against selected targets, collecting the results, analyzing the detected vulnerabilities, and comparing the obtained findings with the expected security weaknesses.

The evaluation focused on the following criteria:

- detection of common web vulnerabilities;
- ability to analyze dynamic web applications;
- quality of result aggregation and deduplication;
- clarity of vulnerability presentation;
- comparison with traditional scanning approaches.

5.2.2 Case Study: OWASP Juice Shop

OWASP Juice Shop was used as a modern vulnerable web application to evaluate the ability of CyberScope to handle dynamic behavior and client-side interactions [17].

This case study is particularly important because Juice Shop relies heavily on modern web technologies and client-side rendering. As a result, it represents a relevant target for testing the BrowserAgent and dynamic crawling capabilities of CyberScope.

The scan results obtained from this target allow the evaluation of the system's ability to discover endpoints, analyze requests, and detect vulnerabilities in a dynamic environment.

5.2.3 Case Study: Altoro Mutual

Altoro Mutual was used as a traditional vulnerable web application to evaluate the system against classic web vulnerabilities [18].

This target allows CyberScope to test its ability to detect issues such as injection flaws, misconfigurations, and insecure application behavior. It also provides a useful comparison point between modern and traditional web application testing scenarios.

5.3 Results Analysis

The results obtained during experimental validation show that CyberScope is capable of coordinating multiple tools and presenting their findings in a structured format.

The system improves the analysis process by grouping related findings, assigning severity levels, and presenting contextual information. This helps security analysts focus on the most relevant vulnerabilities instead of manually reviewing raw outputs from different tools.

5.3.1 True Positives vs False Positives

One of the main objectives of CyberScope is to reduce noise in vulnerability reports. Traditional tools may produce redundant or irrelevant findings, which increases the workload of the analyst.

By applying normalization and deduplication mechanisms, CyberScope reduces repeated findings and improves the readability of results. However, some false positives may still appear, especially when a tool reports a potential issue that requires manual confirmation.

5.3.2 Detection Coverage

CyberScope improves detection coverage by combining multiple tools and dynamic crawling. The use of several scanners increases the chances of identifying different categories of vulnerabilities.

The BrowserAgent contributes to this coverage by discovering API calls and hidden endpoints that may not be visible through static crawling. This is especially useful for modern Single Page Applications.

5.3.3 System Limitations

Despite its advantages, CyberScope still presents some limitations. Some findings require manual validation, especially when the severity or exploitability depends on application-specific context.

In addition, scan duration may increase when multiple tools are executed or when the target application is large and dynamic. The quality of the results also depends on the configuration and reliability of the integrated tools.

5.4 Comparative Evaluation with Existing Tools

Compared to standalone tools such as Nikto, OWASP ZAP, and Burp Suite, CyberScope provides a more integrated approach by combining multiple tools within a unified pipeline [3, 4, 5].

Table 5.1: Comparative Evaluation of CyberScope

Criteria	**Nikto**	**OWASP ZAP**	**Burp Suite**	**CyberScope**
Automation	High	High	Medium	High
Dynamic application support	Limited	Partial	Good	Advanced with Browser-Agent
Multi-tool integration	No	Limited	Limited	Yes
Result normalization	Limited	Limited	Partial	Yes
Target profiling	No	Partial	Partial	Yes
Result visualization	Basic	Moderate	Advanced	Structured dashboard
False positive reduction	Limited	Medium	Medium	Improved through deduplication

This comparison shows that CyberScope does not aim to replace all existing tools. Instead, it enhances their effectiveness by orchestrating them, processing their outputs, and presenting results in a more structured and actionable way.

5.5 Proposed Enhancements

Several improvements can be added to CyberScope in future versions. The first enhancement is the generation of detailed PDF reports, allowing security analysts to export scan results in a professional format.

Another important improvement is the enrichment of vulnerability findings using AI-generated remediation guidance. This would help users better understand the impact of each vulnerability and the recommended corrective actions.

Additional enhancements may include advanced attack graph visualization, CVSS scoring, CVE correlation, user role management, and scheduling of periodic scans [19].

5.6 Future Work

Future work will focus on improving the intelligence and scalability of the platform. This includes extending the BrowserAgent, adding more security tools, improving tool

selection logic, and enhancing the accuracy of vulnerability correlation.

The system could also be extended to support authenticated scanning, more advanced reporting features, and integration with external vulnerability management platforms.

5.7 Conclusion

This chapter evaluated CyberScope through experimental validation and discussed the obtained results. The evaluation showed that the system provides a unified and intelligent approach to web application security auditing.

By combining multi-tool orchestration, dynamic crawling, result normalization, and structured visualization, CyberScope improves the efficiency of vulnerability analysis. However, some limitations remain, particularly regarding manual validation and advanced contextual interpretation, which represent opportunities for future improvement.

Chapter 6

Limitations, Perspectives and Future Work

6.1 Introduction

This final chapter completes the technical evaluation of **CyberScope** by discussing the limits observed during development and validation. It also identifies concrete enhancements that can transform the current prototype into a more complete operational platform.

The objective is not only to list future features, but also to explain why these improvements are relevant in the context of automated web application security auditing.

6.2 Synthesis of Contributions

The main contribution of this project is the integration of several security capabilities into a unified platform. Instead of relying on a single scanner, CyberScope coordinates different tools and processing layers in order to produce more structured results.

The implemented system provides the following contributions:

- a full-stack architecture connecting a React frontend, a Node.js backend, a PostgreSQL database, and a Python scanning engine;
- authenticated access with JWT tokens, password hashing, and user-scoped scan history;
- a scan orchestration layer that communicates with HexStrike and supports scan profiles such as quick, balanced, and global scans;
- protection mechanisms such as URL validation and SSRF prevention before launching scans;
- passive security checks covering missing headers, cookie attributes, exposed files, insecure transport, technology disclosure, and potential secrets;
- a normalization and deduplication engine that transforms raw scanner outputs into structured findings;
- target profile visualization, pipeline diagnostics, scan history, and detailed scan result pages;
- an AI-assisted attack path and risk analysis module based on Ollama and visualized through React Flow [20, 16].

These contributions show that CyberScope goes beyond a simple scanner wrapper. It acts as an orchestration and interpretation layer between security tools and the

analyst.

6.3 Comparative Positioning

Compared with standalone tools, CyberScope focuses on coordination, result processing, and usability. Table 6.1 summarizes this positioning.

Table 6.1: Positioning of CyberScope

Aspect	Traditional Usage	CyberScope Approach
Tool execution	Tools are launched separately by the analyst.	Tools are coordinated through a backend pipeline and scan profile.
Result format	Outputs are often raw logs or tool-specific reports.	Findings are normalized into a common structure with severity and evidence.
Noise handling	The analyst manually removes banners, errors, and duplicates.	Filtering and deduplication reduce low-value findings before visualization.
Target context	Context is usually collected manually.	The target profile groups technologies, ports, endpoints, and risk indicators.
Analysis support	Interpretation depends heavily on expert manual work.	AI-assisted attack scenarios and risk assessments support interpretation.

This positioning confirms that CyberScope complements existing scanners by improving orchestration and interpretation rather than replacing specialized tools.

6.4 Identified Limitations

Despite the progress achieved, the implemented version still presents several limitations.

6.4.1 Dependency on External Tools

CyberScope depends on the availability and reliability of external security tools. If a tool is missing, misconfigured, slow, or returns only an error message, the quality of the scan can be affected. The backend includes diagnostics to detect failed tools, timeouts, and unusable outputs, but it cannot fully compensate for toolchain instability.

6.4.2 Scan Duration and Resource Consumption

Comprehensive scans may take a long time because they launch multiple tools and may explore many endpoints. This is especially true for large applications or dynamic targets. The scan profile mechanism reduces this issue by allowing quick, balanced, and global modes, but further optimization is still required.

6.4.3 False Positives and Manual Confirmation

Normalization and filtering reduce noise, but they do not eliminate the need for manual validation. Some findings depend on the business context, authentication state, user privileges, or exploitability conditions. Therefore, CyberScope should be considered a decision-support platform rather than a fully autonomous security auditor.

6.4.4 Limited Authenticated Target Crawling

The current platform authenticates CyberScope users, but authenticated scanning of target applications remains limited. For real enterprise audits, the scanner should support cookies, tokens, login workflows, and role-based target exploration.

6.4.5 Reporting and Integration

The frontend presents scan results and diagnostics, but professional PDF report generation and integration with external ticketing or vulnerability management platforms remain future work.

6.5 Proposed Enhancements

Several enhancements can improve CyberScope in future versions.

- **Authenticated target scanning:** allow analysts to provide cookies, bearer tokens, or scripted login workflows so that private application areas can be tested.
- **Professional report generation:** export scan results as PDF or DOCX reports including executive summaries, technical details, evidence, remediation, and appendices.
- **CVE and CVSS enrichment:** connect findings to vulnerability databases and compute more consistent risk scores [19].
- **Scheduled scans:** allow periodic scans and comparison between previous and current results.
- **Role management:** introduce roles such as administrator, analyst, and viewer to better control access.
- **Containerized deployment:** provide Docker-based deployment for the backend, frontend, database, and scanning engine.
- **Improved frontend analytics:** add charts for severity distribution, tool performance, scan duration, and recurring vulnerabilities.

6.6 Future Research Directions

Future research can focus on the intelligence of the scanning pipeline. One direction is to improve the BrowserAgent so that it can learn common navigation patterns, handle

authentication flows, and interact with complex forms more effectively. This would increase coverage for Single Page Applications and API-driven systems.

Another research direction is the use of AI for evidence verification. Instead of only generating attack scenarios, the AI layer could help classify findings, identify weak evidence, propose validation steps, and explain why a finding may be a false positive. This would require strict controls to avoid hallucinated conclusions.

CyberScope can also be extended toward DevSecOps integration. In such a workflow, scans could be triggered automatically during deployment, results could be compared between releases, and critical vulnerabilities could create tickets or block releases according to security policies.

Finally, attack path visualization can be improved by linking vulnerabilities, assets, privileges, and potential attacker actions. The MITRE ATT&CK knowledge base can support this direction by providing a structured vocabulary for adversary tactics and techniques [21]. This would help analysts understand not only isolated vulnerabilities, but also how several weaknesses could be chained in a realistic attack scenario.

6.7 Conclusion

This chapter presented the main limitations of CyberScope and identified concrete perspectives for improvement. The system already provides a functional foundation for automated web application security auditing, but several enhancements are needed to make it suitable for broader professional use.

The most important future directions are authenticated target crawling, richer reporting, stronger vulnerability intelligence, better integration with operational workflows, and deeper AI-assisted analysis. These improvements would strengthen CyberScope's value as a practical platform for security analysts.

General Conclusion

This end-of-studies project made it possible to design and implement **CyberScope**, an intelligent web application security auditing platform. The work started from a concrete problem: traditional vulnerability scanners are useful, but their outputs are often fragmented, noisy, and difficult to exploit directly when modern web applications rely on dynamic interfaces, APIs, and heterogeneous technologies.

The proposed solution answers this problem through a complete architecture composed of a React/TypeScript frontend, a Node.js/Express backend, a PostgreSQL database managed through Prisma, and a Python scanning engine based on HexStrike. The platform enables authenticated users to launch scans, select scan profiles, consult scan history, inspect target profiles, review normalized vulnerabilities, and analyze pipeline diagnostics.

On the backend side, several important mechanisms were implemented: JWT authentication, password hashing, user-scoped scan persistence, SSRF protection, communication with the HexStrike engine, passive security checks, vulnerability matching, normalization, deduplication, and noise filtering. On the frontend side, the platform provides scanner, history, details, and attack-path interfaces. The AI layer, connected to Ollama, extends the system by generating attack scenarios and risk assessments from structured vulnerability data.

The validation phase showed that CyberScope can coordinate multiple tools, extract useful findings, reduce irrelevant output, and present results in a format that is more readable than raw scanner logs. The benchmark artifacts demonstrated the detection of representative weaknesses such as SQL injection, cross-site scripting, information disclosure, exposed files, and security misconfigurations. The evaluation also highlighted practical limitations, including dependency on external tool stability, scan duration, partial false positives, and the need for manual confirmation of certain findings.

Overall, CyberScope provides a solid foundation for automated web security auditing. It does not aim to replace expert security analysis, but rather to support it by automating repetitive tasks, correlating results, and improving decision-making. Future improvements may include authenticated target crawling, richer report export, continuous scan scheduling, stronger CVE/CVSS correlation, deeper BrowserAgent behavior, and integration with professional vulnerability management workflows.

References

Bibliography

[1] OWASP Foundation, "Owasp top 10:2021." https://owasp.org/Top10/2021/, 2021. Accessed: 26 April 2026.

[2] Ernst & Young, "Ey tunisia - office location and contact information." https://www.ey.com/en_gl/locations/tunisia, 2026. Accessed: 26 April 2026.

[3] CIRT.net, "Nikto web scanner." https://cirt.net/nikto/, 2026. Accessed: 26 April 2026.

[4] OWASP, "Zed attack proxy documentation." https://www.zaproxy.org/, 2026. Accessed: 26 April 2026.

[5] PortSwigger, "Burp suite." https://portswigger.net/burp, 2026. Accessed: 26 April 2026.

[6] React Team, "React documentation." https://react.dev/, 2026. Accessed: 26 April 2026.

[7] OpenJS Foundation, "Express documentation." https://expressjs.com/, 2026. Accessed: 26 April 2026.

[8] PostgreSQL Global Development Group, "Postgresql documentation." https://www.postgresql.org/docs/, 2026. Accessed: 26 April 2026.

[9] Prisma, "Prisma documentation." https://docs.prisma.io/, 2026. Accessed: 26 April 2026.

[10] ProjectDiscovery, "Nuclei documentation." https://docs.projectdiscovery.io/opensource/nuclei, 2026. Accessed: 26 April 2026.

[11] SQLMap Project, "Sqlmap automatic sql injection tool." https://sqlmap.org/, 2026. Accessed: 26 April 2026.

[12] Nmap Project, "Nmap documentation." https://nmap.org/docs.html, 2026. Accessed: 26 April 2026.

[13] M. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," RFC 7519, Internet Engineering Task Force, 2015. Available at: https://datatracker.ietf.org/doc/html/rfc7519. Accessed: 26 April 2026.

[14] OWASP Foundation, "Server-side request forgery prevention cheat sheet." https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html, 2026. Accessed: 26 April 2026.

[15] TanStack, "Tanstack query documentation." https://tanstack.com/query/docs, 2026. Accessed: 26 April 2026.

[16] XYFlow, "React flow documentation." https://reactflow.dev/, 2026. Accessed: 26 April 2026.

[17] OWASP Foundation, "Owasp juice shop project." https://owasp.org/www-project-juice-shop, 2026. Accessed: 26 April 2026.

[18] HCL Software, "Altoro mutual demonstration website." https://demo.testfire.net/, 2026. Accessed: 26 April 2026.

[19] Forum of Incident Response and Security Teams, "Common vulnerability scoring system." https://www.first.org/cvss/, 2026. Accessed: 26 April 2026.

[20] Ollama, "Ollama api documentation." https://docs.ollama.com/api, 2026. Accessed: 26 April 2026.

[21] MITRE, "Mitre att&ck knowledge base." https://attack.mitre.org/, 2026. Accessed: 26 April 2026.

Chapter A

Technical Appendix

A.1 Main API Endpoints

CyberScope exposes a REST API used by the frontend dashboard. The most important endpoints are summarized in Table A.1.

Table A.1: Main backend API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Create a new user account and return a JWT session token.
POST	/api/auth/login	Authenticate an existing user using email and password.
POST	/api/auth/guest	Create a restricted guest session for interface exploration.
GET	/api/auth/me	Retrieve the current authenticated user.
POST	/api/scans	Launch a new scan for an authenticated user.
GET	/api/scans	Retrieve the authenticated user's scan history.
GET	/api/scans/:scanId	Retrieve scan details, findings, metadata, and diagnostics.
GET	/api/vulnerabilities	Retrieve known and discovered vulnerabilities.
POST	/api/attacks/ai-generate	Generate an AI-assisted attack scenario for a vulnerability.
POST	/api/attacks/ai-risk	Generate an AI-assisted risk assessment for a scan.

A.2 Environment Variables

The backend uses environment variables to configure database access, authentication, scanner communication, and AI services. Table A.2 lists the main variables used by the implementation.

Table A.2: Important backend environment variables

Variable	Role

DATABASE_URL	PostgreSQL connection string used by Prisma.
JWT_SECRET	Secret key used to sign and verify JWT tokens.
JWT_EXPIRES_IN	Session lifetime for authenticated users.
HEXSTRIKE_BASE_URL	Base URL of the Python HexStrike scanning engine.
HEXSTRIKE_TIMEOUT_MS	Maximum duration allowed for a scanner request.
HEXSTRIKE_OBJECTIVE	Default scanning objective sent to the engine.
HEXSTRIKE_MAX_TOOLS	Maximum number of tools executed by default.
HEXSTRIKE_ALLOW_LOCAL_SCAN	Enables local scanning during controlled testing.
OLLAMA_BASE_URL	Base URL of the local Ollama service.
OLLAMA_MODEL	LLM model used for attack scenario and risk generation.
OLLAMA_TIMEOUT_MS	Maximum duration allowed for AI generation requests.

A.3 Validation Scripts and Artifacts

The project contains several scripts and JSON artifacts used during validation. The most relevant elements are:

- `scripts/verify-normalization-benchmark.js`: validates the normalization and noise filtering logic on controlled scanner outputs.
- `scripts/process-benchmark.js`: processes captured HexStrike outputs and generates a compact benchmark summary.
- `benchmark_final.json`: contains normalized findings extracted from a representative vulnerable target.
- `validation_result.json`: contains a local validation run with tool execution diagnostics.
- `debug_scan_output.json`: contains a captured scan output used to test normalization robustness.